

## **Chapter -2**

# **Lexical Analysis/Scanner**

# Session1

- Lexical Analyzer
- The Role of the Lexical Analyzer
- What is token?
- Tokens, Patterns and Lexemes
- Token Attributes
- Lexical Errors
- Input Buffering
- Specification of Tokens
- Recognition of Tokens

# Lexical Analyzer

- **Lexical analysis** is the process of converting a sequence of characters into a sequence of tokens.
- A program or function which performs lexical analysis is called a **lexical analyzer, Lexer, or Scanner**.
- A **lexer** often exists as a single function which is called by a parser (syntax Analyzer) or another function.
- **The Role of the Lexical Analyzer**
  - Read the input characters
  - Grouping them into lexemes
  - Output a sequence of tokens
  - Others:
    - *Macros expansion*: If the source program uses a macro preprocessor, the expansion of macros may also be performed by the lexical analyzer

# The Role of the Lexical Analyzer con...

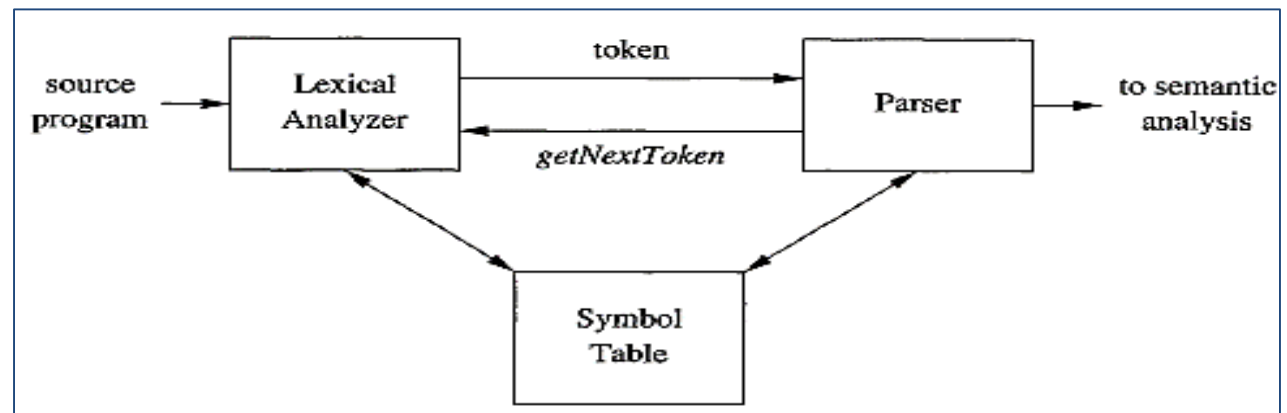
- *Stripping out comments* and white spaces and other separators
- *Correlating error messages* with the source program

Egg. keep track of number of newline characters seen for associating a line number with each error message

- *Pass token to parser*

- *Interact with:*

- Parser: for syntax analysis
- Symbol table: for gathering information regarding to kind of identifier to assist it in determining the proper token it must pass to the parser



# Lexical Analyzer ....

- Sometimes Lexical Analyzers are divided into a cascade of two phases; **Scanning** and **Lexical analysis proper**
- **Scanning** consists of the simple processes that do not require tokenization of the input.
  - (egg. deletion of comments and compaction of consecutive whitespace characters into one).
- **Lexical analysis proper** is the more complex portion, where the scanner produces the sequence of tokens as output.

# Lexical Analyzer ....

- Advantages of separating **lexical analysis** from **other phases** are:

- Simpler design - Separate the two analysis
  - High cohesion / Low coupling
- Improve maintainability
- Compiler efficiency
  - Liberty to apply specialized techniques that serves only lexical tasks, not the whole parsing
  - Speedup reading input characters using specialized buffering techniques
- Compiler portability (e.g. Linux to Windows)
- Input device specific peculiarities are restricted to the lexical analyzer
  - Enable integration of third-party Lexers [lexical analysis tool]

# What is token?

- A token is a pair consisting of a token name and an optional attribute value.
- The token name is an abstract symbol representing a kind of lexical unit
- Tokens correspond to sets of strings.
  - *Identifier*: strings of letters or digits, starting with a letter
  - *Integer*: a non-empty string of digits; 123, 123.45
  - *Keyword*: “else” or “if” or “begin” or ...
  - *Whitespace*: a non-empty sequence of blanks, newlines, & tabs
  - *Symbols*: +, -, \*, /, =, <, >, ->, ...
  - *Char (string) literals*: “Hello”, ‘c’
  - Comments
  - Terminal symbols in a grammar
- The token names are the input symbols that the parser processes.

# What is token? ...Example

## ■ What do we want to do?

---

- Example:

if (i == j)

  Z = 0;

else

  Z = 1;

- The input is just a string of characters:

\t if (i == j) \n \t \t z = 0;\n \t else \n \t \t z = 1;

- Goal: Partition input string into substrings
  - Where the substrings are tokens
  - Tokens: if, (, i, ==, j, ), z, =, 0, ,, else, 1, ...



# Tokens, Patterns and Lexemes

- Lexeme: Actual sequence of characters that matches a pattern and has a given token class.
  - Examples: Name, Data, x, 345,2,0,629,....
- Token
  - A *classification* for a common set of strings
  - Examples: Identifier, Integer, Float, Assign, LeftParen, RightParen,....
    - One token for all identifiers:
      - egg. **id** is token for different identifiers: a, b, c , ...
- Pattern: a description of the form that the lexemes of a token may take.
  - The rules that characterize the set of strings for a token
  - Examples:  $[0-9]^+$ 
    - identifier:  $([a-z] | [A-Z]) ([a-z] | [A-Z] | [0-9])^*$

# Tokens, Patterns and Lexemes: Example

| TOKEN      | INFORMAL DESCRIPTION                  | SAMPLE LEXEMES      |
|------------|---------------------------------------|---------------------|
| if         | characters i, f                       | if                  |
| else       | characters e, l, s, e                 | else                |
| comparison | < or > or <= or >= or == or !=        | <=, !=              |
| id         | letter followed by letters and digits | pi, score, D2       |
| number     | any numeric constant                  | 3.14159, 0, 6.02e23 |
| literal    | anything but ", surrounded by "'s     | "core dumped"       |

# Token Attributes

- More than one lexeme can match a pattern

---

    - the lexical analyzer must provide the information about the particular lexeme that matched.
    - For example, the pattern for token number matches both 0, 1, 934, ...
  - But code generator must know which lexeme was found in the source program.
    - Thus, the lexical analyzer returns to the parser **a token name** and **an attribute value**
  - For each lexeme the following type of output is produced  
*(token-name, attribute-value)*
- ➔ *attribute-value points to an entry in the symbol table for this token*

# Example of Attribute Values

- $E = M * C ** 2$  [Fortran source code]
- *Token names and associated attribute values:*
  - <id, pointer to symbol table entry for E>
  - <assign\_op>
  - <id, pointer to symbol table entry for M>
  - <mult\_op>
  - <id, pointer to symbol table entry for C>
  - <exp\_op>
  - <number, integer value 2>

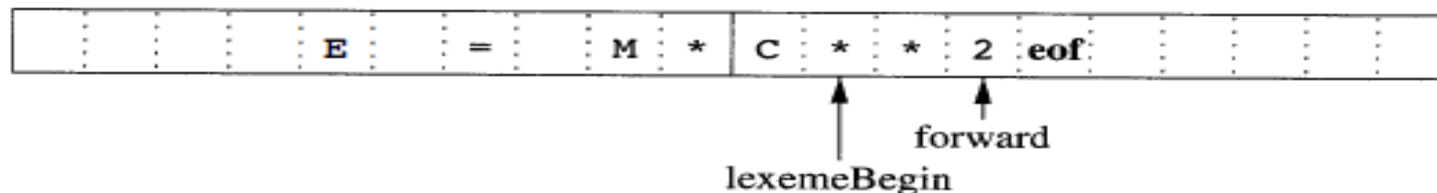
# Lexical Errors

- Lexical analyzer can't detect all errors – needs other components
- In what situations do errors occur?
  - When no patterns for tokens matches any prefix of the remaining input (character sequence).
- However look egg.: `fi(a==f(x)) ... in C program`
  - A lexical analyzer cannot tell whether **fi** is a misspelling of the keyword **if** or an undeclared function identifier. Since **fi** is a valid lexeme for the token **id**, the lexical analyzer must return the token **id** to the parser.
  - so this error will generate by next phases of compiler
- Possible error recovery actions:
  - Panic mode: successive characters are ignored until we reach to a well formed token
  - Deleting or Inserting Input Characters
  - Replacing a character by another character
  - Transposing two adjacent characters. Egg. `fi` to `if`
  - Or, skip over to next separator to ignore problem

# Input Buffering

- ✧ There are some ways that the task of reading the source program can be speeded
- ✧ This task is made difficult by the fact that we often have to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme
  - ☞ In C language: we need to look after -, = or < to decide what token to return
  - ☞ We shall introduce a two-buffer scheme that handles **large look-aheads** safely
- ✧ We then consider an improvement involving sentinels that saves time checking for the ends of buffers
- ✧ Because of the amount of time taken to process characters and the large number of characters that must be processed during compilation of a large source program, specialized buffering techniques have been developed to reduce the amount of overhead to process a single input character
  - ☞ An important scheme involves **two buffers** that are alternatively reloaded, as shown in the

## Contd.



- ✧ Each buffer is of the same size **N**, and **N** is usually the size of a disk block, e.g., 4096 bytes
- ✧ Using one system read command we can read **N** characters into a buffer, rather than using one system call per character
  - ☞ If fewer than **N** characters remain in the input file, then a special character, represented by **eof**, marks the end of the source file and is different from any possible character of the source program
- ✧ Two pointers to the input are maintained:
  1. **Pointer lexemeBegin**, marks the beginning of the current lexeme, whose extent we are attempting to determine
  2. **Pointer forward** scans ahead until a pattern match is found

## Contd.

- ✚ Once the lexeme is determined, **forward** is set to the character at its right end (involves retracting)
  - ✚ Then, after the lexeme is recorded as an attribute value of the token returned to the parser, **lexemeBegin** is set to the character immediately after the lexeme just found
- ✚ Advancing forward requires that we first test whether we have reached the end of one of the buffers, and if so, we must reload the other buffer from the input, and move forward to the beginning of the newly loaded buffer

## Sentinels

- ✚ If we use the previous scheme, we must check each time we advance forward, that we have not moved off one of the buffers; if we do, then we must also reload the other buffer
  - ✚ Thus, for each character read, we must make **two tests**: **one for the end of the buffer**, and **one to determine which character is read**
  - ✚ We can combine the buffer-end test with the test for the current character if we extend each buffer to hold sentinel character at the end



## Contd.

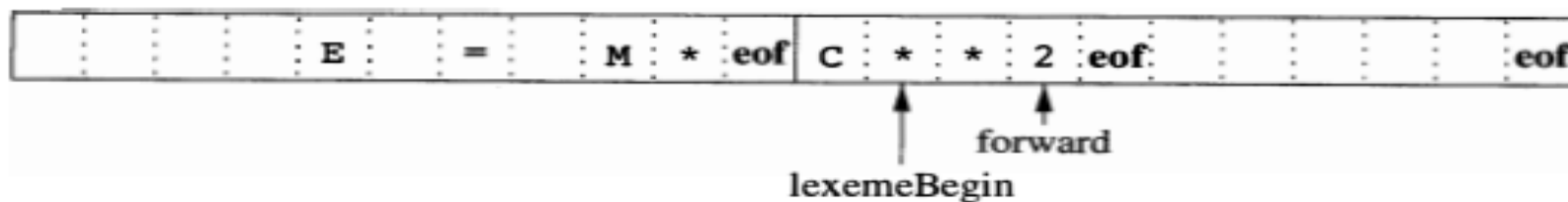


Fig. Sentinels at the end of each buffer

✧ The **sentinel** is a special character that cannot be part of the source program, and a natural choice is the character **eof**

✧ Note that eof retains its use as a marker for the end of the entire input

✧ Any eof that appears other than at the end of buffer means the input is at an end

✧ The above Figure shows the same arrangement as Figure in slide no. 16, but

17 with the sentinels added.

```
Switch (*forward++) {  
    case eof:  
        if (forward is at end of first buffer) {  
            reload second buffer;  
            forward = beginning of second buffer;  
        }  
        else if {forward is at end of second buffer) {  
            reload first buffer;\n            forward = beginning of first buffer;  
        }  
        else /* eof within a buffer marks the end of input */  
            terminate lexical analysis;  
            break;  
        cases for the other characters;  
    }  
}
```

# Specification of Tokens

- Two issues in lexical analysis.
  - *How to specify tokens?* And
  - *How to recognize the tokens giving a token specification?* (i.e. how to implement the **nexttoken( )** routine)?
- How to specify tokens:
  - Tokens are specified by **regular expressions**.
- ***Regular Expressions:***
  - Represent patterns of strings of characters
  - Regular expressions are important part in specifying lexeme patterns.
  - Each pattern matches a set of strings, so regular expressions will serve as names for sets of strings.
  - cannot express all possible patterns, they are very effective in specifying those types of patterns that we actually need for tokens.
  - The set of strings generated by a regular expression **r** is as **L(r)**

# Strings and Alphabets

- *alphabet* : any finite set of symbols.
  - E.g. {a, b, c}, {0, 1} binary alphabet
- A *string/sentence/word* over an alphabet is a finite sequence of symbols drawn from that alphabet.
- Length of string  $s$  is denoted  $|s|$
- Operations.
  - concatenation of strings  $x$  and  $y$ :  $xy$ ;  $\epsilon x = x$   $\epsilon x = x$
  - exponential :  $S^0 = \epsilon$ ,  $S^1 = S$ ,  $S^2 = SS$ , ...  $S^i = S^{i-1}S$ ,  $i > 0$

# Languages

- A language is any countable set of strings over some fixed alphabet.

## Alphabet

$\{0,1\}$

$\{a, b, c\}$

$\{A...Z\}$

$\{A...Z, a...z, 0...9, \\ +, -, ..., <, >, ...\}$

Special Languages:

## Language

$\{0,10,100,1000,10000,...\}$

$\{0,1,100,000,111,...\}$

$\{abc, aabbcc, aaabbbccc,...\}$

$\{TEE,FORE,BALL... \}$

$\{FOR,WHILE,GOTO... \}$

$\{\text{All legal PASCAL progs}\}$

$\{\text{All grammatically correct English Sentences}\}$

$\Phi$  – EMPTY LANGUAGE

$\epsilon$  – contains empty string  $\epsilon$  only

# Operations on Languages

| OPERATION                       | DEFINITION AND NOTATION                                                   |
|---------------------------------|---------------------------------------------------------------------------|
| <i>Union of L and M</i>         | $L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$ |
| <i>Concatenation of L and M</i> | $LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$     |
| <i>Kleene closure of L</i>      | $L^* = \bigcup_{i=0}^{\infty} L^i$                                        |
| <i>Positive closure of L</i>    | $L^+ = \bigcup_{i=1}^{\infty} L^i$                                        |

$$L = \{A, B, C, D\}$$

$$D = \{1, 2, 3\}$$

$$L \cup D = \{A, B, C, D, 1, 2, 3\}$$

$$LD = \{A1, A2, A3, B1, B2, B3, C1, C2, C3, D1, D2, D3\}$$

$$L^2 = \{AA, AB, AC, AD, BA, BB, BC, BD, CA, CB, CD, DA, DB, DD\}$$

$$L^4 = L^2 L^2 = ??$$

$$L^* = \{\text{All possible strings of } L \text{ plus } \epsilon\}$$

$$L^+ = L^* - \epsilon$$

$$L(L \cup D) \text{ set of all strings of letters and digits beginning with a letter. \& each length} = 2$$

$$L(L \cup D)^* \text{ set of all strings of letters \& digits starting with a letter. \& length } \geq 1$$

# Regular Expressions

## ■ Formal definition of Regular expression:

---

- Given an alphabet  $\Sigma$  ,
- (1)  $\epsilon$  is a regular expression that denote  $\{\epsilon\}$ , the set that contains the empty string and  $L\{\epsilon\} = \{\epsilon\}$
- (2) For each  $a \in \Sigma$  ,  $a$  is a regular expression denote  $\{a\}$ , the set containing the string  $a$  and  $L\{a\} = \{a\}$
- (3)  $r$  and  $s$  are regular expressions, then
  - $(r \mid s)$  is a regular expression denoting  $L(r) \cup L(s)$
  - $(rs)$  is a regular expression denoting  $L(r) L(s)$
  - $(r)^*$  is a regular expression denoting  $(L(r))^*$
- Regular expression is defined together with the language it denotes.
- Regular expressions are used to denote ***regular languages***

# Regular Expressions: Example

Let  $\Sigma = \{a, b\}$ .

1. The regular expression  $\mathbf{a|b}$  denotes the language  $\{a, b\}$ .
2.  $\mathbf{(a|b)(a|b)}$  denotes  $\{aa, ab, ba, bb\}$ , the language of all strings of length two over the alphabet  $\Sigma$ . Another regular expression for the same language is  $\mathbf{aa|ab|ba|bb}$ .
3.  $\mathbf{a^*}$  denotes the language consisting of all strings of zero or more  $a$ 's, that is,  $\{\epsilon, a, aa, aaa, \dots\}$ .
4.  $\mathbf{(a|b)^*}$  denotes the set of all strings consisting of zero or more instances of  $a$  or  $b$ , that is, all strings of  $a$ 's and  $b$ 's:  $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$ . Another regular expression for the same language is  $\mathbf{(a^*b^*)^*}$ .
5.  $\mathbf{a|a^*b}$  denotes the language  $\{a, b, ab, aab, aaab, \dots\}$ , that is, the string  $a$  and all strings consisting of zero or more  $a$ 's and ending in  $b$ .

# Regular Expressions: Algebraic Properties

| LAW                              | DESCRIPTION                                  |
|----------------------------------|----------------------------------------------|
| $r s = s r$                      | $ $ is commutative                           |
| $r (s t) = (r s) t$              | $ $ is associative                           |
| $r(st) = (rs)t$                  | Concatenation is associative                 |
| $r(s t) = rs rt; (s t)r = sr tr$ | Concatenation distributes over $ $           |
| $\epsilon r = r\epsilon = r$     | $\epsilon$ is the identity for concatenation |
| $r^* = (r \epsilon)^*$           | $\epsilon$ is guaranteed in a closure        |
| $r^{**} = r^*$                   | $*$ is idempotent                            |

- All Strings in English alphabet that start with “tab” or end with “bat”:  
 $\text{tab}\{A,\dots,Z,a,\dots,z\}^*|\{A,\dots,Z,a,\dots,z\}^*\text{bat}$
- All Strings in English alphabet in Which  $\{1,2,3\}$  exist in ascending order:  
 $\{A,\dots,Z\}^*1\{A,\dots,Z\}^*2\{A,\dots,Z\}^*3\{A,\dots,Z\}^*$



# RE's: Examples

□  $L(\mathbf{01}) = ?$

□  $L(\mathbf{01} \mid \mathbf{0}) = ?$

□  $L(\mathbf{0(1 \mid 0)}) = ?$

□ Note order of precedence of operators.

□  $L(\mathbf{0^*}) = ?$

□  $L((\mathbf{0 \mid 10})^*(\epsilon \mid \mathbf{1})) = ?$

# RE's: Examples

□  $L(\mathbf{01}) = \{01\}$ .

□  $L(\mathbf{01} \mid \mathbf{0}) = \{01, 0\}$ .

□  $L(\mathbf{0(1 \mid 0)}) = \{01, 00\}$ .

□ Note order of precedence of operators.

□  $L(\mathbf{0^*}) = \{\epsilon, 0, 00, 000, \dots\}$ .

□  $L((\mathbf{0 \mid 10})^*(\epsilon \mid \mathbf{1})) = \text{all strings of 0's and 1's without two consecutive 1's.}$

# RE's: Examples (more)

- 1-  $a \mid b = ?$
- 2-  $(a \mid b)a = ?$
- 3-  $(ab) \mid \varepsilon = ?$
- 4-  $((a \mid b)a)^* = ?$

## Reverse

- 1 - Even binary numbers =?
- 2 - An alphabet consisting of just three alphabetic characters:  $\Sigma = \{a, b, c\}$ . Consider the set of all strings over this alphabet that contains exactly one b.

# RE's: Examples (more)

- 1-  $a \mid b = \{a, b\}$
- 2-  $(a \mid b)a = \{aa, ba\}$
- 3-  $(ab) \mid \varepsilon = \{ab, \varepsilon\}$
- 4-  $((a \mid b)a)^* = \{\varepsilon, aa, ba, aaaa, baba, \dots\}$

## Reverse

- 1 - Even binary numbers  $(0 \mid 1)^*0$
- 2 - An alphabet consisting of just three alphabetic characters:  $\Sigma = \{a, b, c\}$ . Consider the set of all strings over this alphabet that contains exactly one b.  
 $(a \mid c)^*b(a \mid c)^* \quad \{b, abc, abaca, baaaac, ccbaca, cccccb\}$

# Exercises

Describe the languages denoted by the following regular expressions:

- 1-  $a(a|b)^*a$
- 2-  $((\epsilon|a)b^*)^*$
- 3-  $(a|b)^*a(a|b)(a|b)$
- 4-  $a^*ba^*ba^*ba^*$

# Regular Definition

- Gives names to regular expressions to construct more complicated regular expressions.
- If  $\Sigma$  is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form:
  - $d_1 \rightarrow r_1$ ,
  - $d_2 \rightarrow r_2$ ,
  - $d_3 \rightarrow r_3$ ,
  - ...
- where:
  - Each  $d_i$  is a new symbol, not in  $\Sigma$  and not the same as any other of the  $d$ 's
  - Each  $r_i$  is a regular expression over the alphabet  $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

# Regular Definition: Example

- Example

- **C** identifiers are strings of letters, digits, and underscores. The regular definition for the language of C identifiers.
  - *Letter*  $\rightarrow A \mid B \mid C \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid \_$
  - *digit*  $\rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$
  - *id*  $\rightarrow \text{letter}(\text{letter} \mid \text{digit})^*$
- Unsigned numbers (integer or floating point) are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.
- The regular definition
  - *digit*  $\rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$
  - *digits*  $\rightarrow \text{digit} \text{ digit}^*$
  - *optionalFraction*  $\rightarrow .\text{digits} \mid \varepsilon$
  - *optionalExponent*  $\rightarrow (E(+ \mid - \mid \varepsilon) \text{ digits}) \mid \varepsilon$
  - *number*  $\rightarrow \text{digits optionalFraction optionalExponent}$

# Short hand notations/extensions of regular expression

- “+”: one or more  $r^* = r^+ | \epsilon$  and  $r^+ = rr^* = r^*r$
- “?”: zero or one  $r? = r | \epsilon$
- [range]: Character classes/set range of characters  $[a_1 | a_2 | \dots | a_n] = [a_1 - a_n]$
- Egg.  $[A-Z] = A | B | C | \dots | Z$  egg2.  $[abc] = a | b | c$
- Example
  - C variable name:  $[A-Za-z][A-Za-z0-9]^*$

## ■ Write regular definitions for

- Phone numbers of form (510) 643-1481
  - *number*  $\rightarrow$  ‘(’ *area* ‘)’ *exchange* ‘-’ *phone*,
  - *phone*  $\rightarrow$  *digit*<sup>4</sup>,
  - *exchange*  $\rightarrow$  *digit*<sup>3</sup>,
  - *area*  $\rightarrow$  *digit*<sup>3</sup>,
  - *digit*  $\rightarrow \{0, 1, 2, 3, \dots, 9\}$
- Email Addresses (**Exercise**)



# Short hand notations/extensions of regular expression

- *Some useful regular expressions*

| EXPRESSION                                 | MATCHES                                                                 | EXAMPLE              |
|--------------------------------------------|-------------------------------------------------------------------------|----------------------|
| <code>c</code>                             | the one non-operator character <code>c</code>                           | <code>a</code>       |
| <code>\c</code>                            | character <code>c</code> literally                                      | <code>\*</code>      |
| <code>"s"</code>                           | string <code>s</code> literally                                         | <code>"**"</code>    |
| <code>.</code>                             | any character but newline                                               | <code>a.*b</code>    |
| <code>^</code>                             | beginning of a line                                                     | <code>^abc</code>    |
| <code>\$</code>                            | end of a line                                                           | <code>abc\$</code>   |
| <code>[s]</code>                           | any one of the characters in string <code>s</code>                      | <code>[abc]</code>   |
| <code>[^s]</code>                          | any one character not in string <code>s</code>                          | <code>[^abc]</code>  |
| <code>r*</code>                            | zero or more strings matching <code>r</code>                            | <code>a*</code>      |
| <code>r+</code>                            | one or more strings matching <code>r</code>                             | <code>a+</code>      |
| <code>r?</code>                            | zero or one <code>r</code>                                              | <code>a?</code>      |
| <code>r{m,n}</code>                        | between <code>m</code> and <code>n</code> occurrences of <code>r</code> | <code>a[1,5]</code>  |
| <code>r<sub>1</sub>r<sub>2</sub></code>    | an <code>r<sub>1</sub></code> followed by an <code>r<sub>2</sub></code> | <code>ab</code>      |
| <code>r<sub>1</sub>   r<sub>2</sub></code> | an <code>r<sub>1</sub></code> or an <code>r<sub>2</sub></code>          | <code>a b</code>     |
| <code>(r)</code>                           | same as <code>r</code>                                                  | <code>(a b)</code>   |
| <code>r<sub>1</sub>/r<sub>2</sub></code>   | <code>r<sub>1</sub></code> when followed by <code>r<sub>2</sub></code>  | <code>abc/123</code> |

# Regular Grammars

- Basically *regular grammars* are used to represent *regular languages*
- The Regular Grammars are either left or right linear:
  - **Right Regular Grammars:**
    - **Rules of the forms:**  $A \rightarrow \varepsilon$ ,  $A \rightarrow a$ ,  $A \rightarrow aB$ ,  $A, B$ : variables and  $a$ : terminal
  - **Left Regular Grammars:**
    - **Rules of the forms:**  $A \rightarrow \varepsilon$ ,  $A \rightarrow a$ ,  $A \rightarrow Ba$ ;  $A, B$ : variables and  $a$ : terminal
  - **Example:**  $S \rightarrow aS \mid bA$ ,  $A \rightarrow cA \mid \varepsilon$ 
    - This grammar produces the language produced by the regular expression  $a^*bc^*$

# Recognition of tokens

- Recognition of tokens is the second issue in lexical analysis
  - How to recognize the tokens giving a token specification?
- Given the grammar of branching statement:

$$\begin{array}{lll} stmt & \rightarrow & \text{if } expr \text{ then } stmt \\ & | & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & | & \epsilon \\ expr & \rightarrow & term \text{ relop } term \\ & | & term \\ term & \rightarrow & \text{id} \\ & | & \text{number} \end{array}$$

- The terminals of the grammar, which are **if**, **then**, **else**, **relop**, **id**, and **number**, are the names of tokens

# Recognition of tokens...

- ❑ The patterns for the given tokens are described using regular definitions as:

$digit \rightarrow [0-9]$   
 $digits \rightarrow digit^+$   
 $number \rightarrow digits ( . digits ) ? ( E [+-]? digits ) ?$   
 $letter \rightarrow [A-Za-z]$   
 $id \rightarrow letter ( letter \mid digit )^*$   
 $if \rightarrow if$   
 $then \rightarrow then$   
 $else \rightarrow else$   
 $relop \rightarrow < \mid > \mid <= \mid >= \mid = \mid <>$

Note that

$r?$  is equivalent to  $r|\epsilon$

- The lexical analyzer also has the job of stripping out whitespace, by recognizing the "token" **ws** defined by:

$ws \rightarrow ( \text{blank} \mid \text{tab} \mid \text{newline} )^+$

# Recognition of tokens...

- ❑ The table shows *which token name is returned to the parser and what attribute value* for each lexeme or family of lexemes.

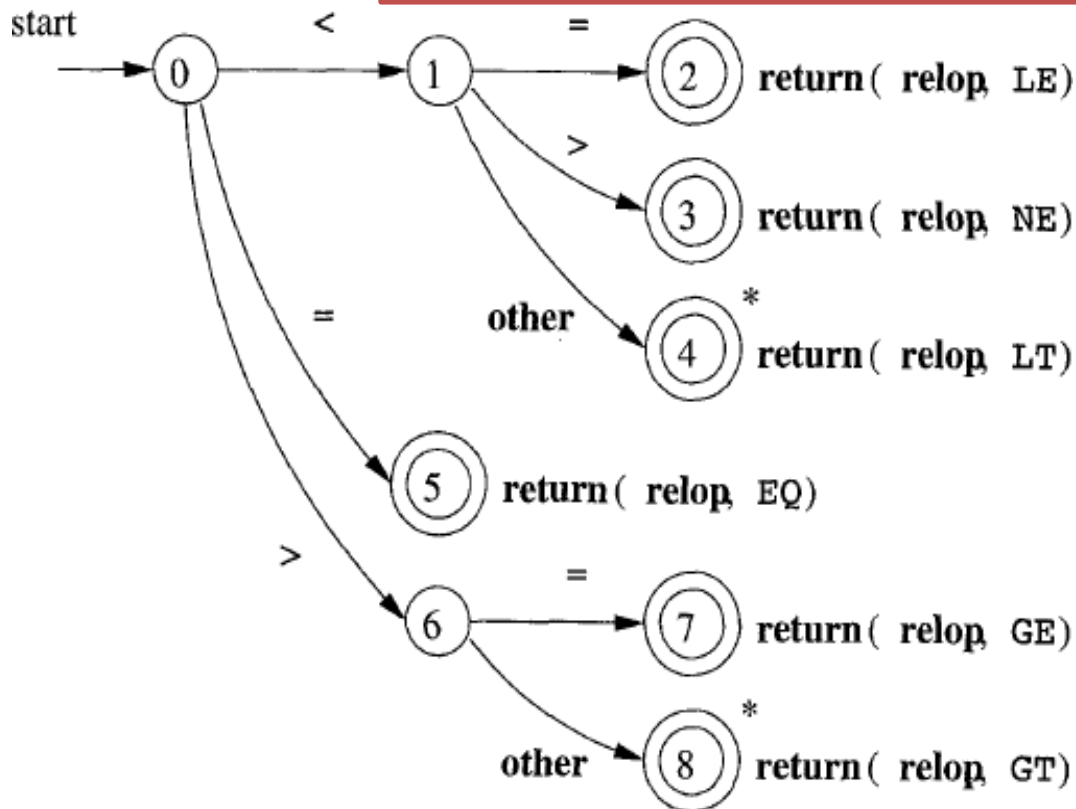
| LEXEMES           | TOKEN NAME    | ATTRIBUTE VALUE        |
|-------------------|---------------|------------------------|
| Any <i>ws</i>     | —             | —                      |
| <b>if</b>         | <b>if</b>     | —                      |
| <b>then</b>       | <b>then</b>   | —                      |
| <b>else</b>       | <b>else</b>   | —                      |
| Any <i>id</i>     | <b>id</b>     | Pointer to table entry |
| Any <i>number</i> | <b>number</b> | Pointer to table entry |
| <b>&lt;</b>       | <b>relop</b>  | LT                     |
| <b>&lt;=</b>      | <b>relop</b>  | LE                     |
| <b>=</b>          | <b>relop</b>  | EQ                     |
| <b>&lt;&gt;</b>   | <b>relop</b>  | NE                     |
| <b>&gt;</b>       | <b>relop</b>  | GT                     |
| <b>&gt;=</b>      | <b>relop</b>  | GE                     |

# Recognition of tokens...

## ■ Transition diagrams

- As intermediate step in the construction of a lexical analyzer, we first convert patterns into stylized flowcharts, called "*transition diagrams*"

### Example Transition diagram for **relop**



- \* is used to indicate that we must retract the input/forward pointer one position (back)
- i.e., the lexeme does not include the symbol that got us to the accepting state

# Recognition of tokens...

- Sketch of implementation of **relop** transition diagram

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

# Recognition of tokens...

## Mapping transition diagrams into C code

```
Token nexttoken( ) {  
    while (1) {  
        switch (state) {  
            case 0:  
                c = nextchar();  
                if (c is white space) state = 0;  
                else if (c == '<') state = 1;  
                else if (c == '=') state = 5;  
                ...  
            case 1:  
                c = nextchar();  
                if(c=='=') go to state 2  
                return;
```

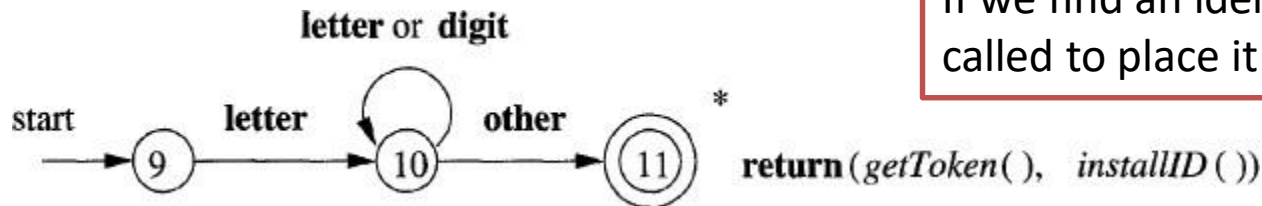
```
        else if(c=='>')  
            goto state 3  
        retrun;  
        else  
            retract ; return;  
        if (isletter( c) ) state = 10;  
        else state =fail();  
            break;  
            case 10: ....  
            case 11: retract(1);  
            insert(id);  
            return;  
        } } }
```



# Recognition of tokens...

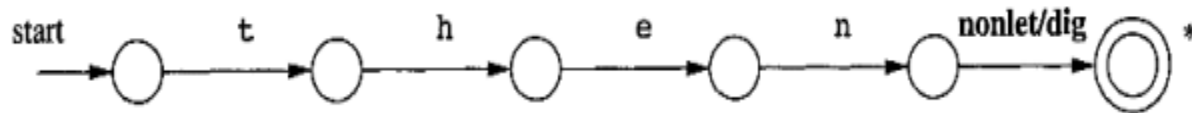
## ■ Recognition of Reserved Words and Identifiers

- Recognizing keywords and identifiers presents a problem.
  - keywords like **if** or **then** are reserved but they look like identifiers
- There are two ways that we can handle
  - install the reserved words in the symbol table initially.



If we find an identifier, *installID* is called to place it in the symbol table.

- Create separate transition diagrams for each keyword;
  - the transition diagram for the reserved word then



NB: A non-letter/digit is used to test any character that cannot be the continuation of an identifier

# Session2

---

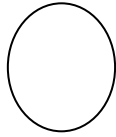
- Finite Automata
- Nondeterministic Finite Automata
- Deterministic Finite Automata
- From Regular Expressions to Automata
- Conversion of NFA to DFA
- State minimization
- Design of a Lexical-Analyzer Generator
- Implementing Scanners

# Finite Automata

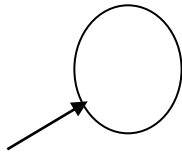
- At the heart of the transition diagram is the formalism known as *finite automata*
- A *finite automaton* is a *finite-state transition diagram* that can be used to model the *recognition* of a *token type* specified by a *regular expression*
- Finite automata are *recognizers*; they simply say "yes" or "no" about each possible input string.
- A finite automaton can be a
  - *Non-deterministic* finite automaton (NFA)
  - *Deterministic* finite automaton (DFA)
- DFA & NFA are capable of recognizing the same languages

# Finite Automata State Graphs

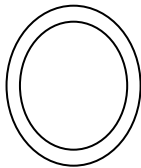
A state



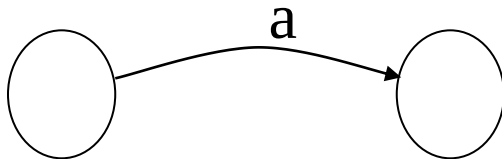
The start state  
(Initial State)



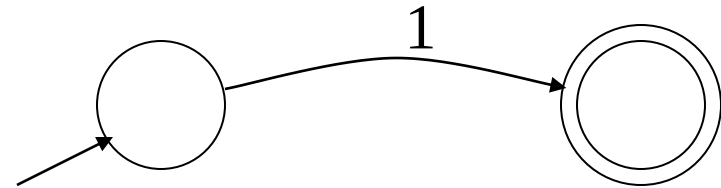
Accepting state  
(Final State)



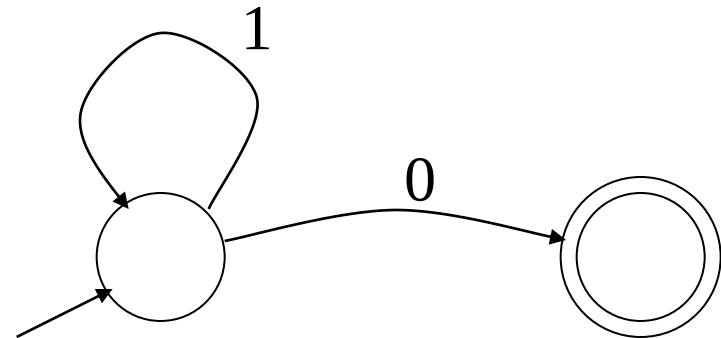
A Transition



**Example 1:** A finite automaton that accepts only “1”



**Example 2:** A finite automaton accepting any number of 1's followed by a single 0



**Q:** Check that “1110” is accepted but “1100...” is not?

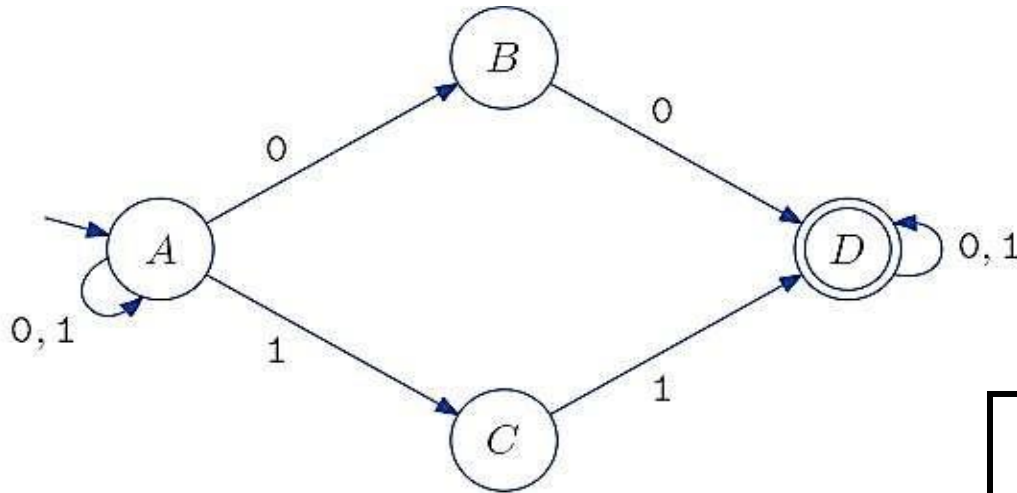
# Nondeterministic Finite Automata

A *nondeterministic finite automaton* (NFA) consists of:

1. A finite set of states  $S$ .
2. A set of input symbols  $\Sigma$ , the *input alphabet*. We assume that  $\epsilon$ , which stands for the empty string, is never a member of  $\Sigma$ .
3. A *transition function* that gives, for each state, and for each symbol in  $\Sigma \cup \{\epsilon\}$  a set of *next states*.
4. A state  $s_0$  from  $S$  that is distinguished as the *start state* (or *initial state*).
5. A set of states  $F$ , a subset of  $S$ , that is distinguished as the *accepting states* (or *final states*).

# NFA: Transition Graph and Transition tables

- An NFA can be diagrammatically represented by a labeled directed graph called a transition graph



The set of states = {A,B, C, D}  
Input symbol = {0, 1}  
Start state is A, accepting state is D

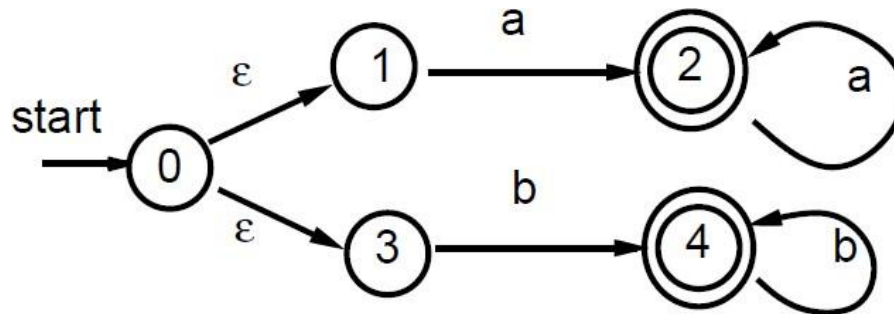
- Transition graph shown above can be represented by transition table on the right side

| State | Input Symbol |       |
|-------|--------------|-------|
|       | 0            | 1     |
| A     | {A,B}        | {A,C} |
| B     | {D}          | --    |
| C     | --           | {D}   |
| D     | {D}          | {D}   |

# Nondeterministic Finite Automata...

## ■ Acceptance of Input Strings by Automata

- An NFA *accepts* an input string *str* **iff** there is *some* path in the finite-state transition diagram from the *start state* to some *final state* such that the *edge labels* along this path spell out *str*
- The *language* recognized by an NFA is *the set of strings* it accepts
- Example

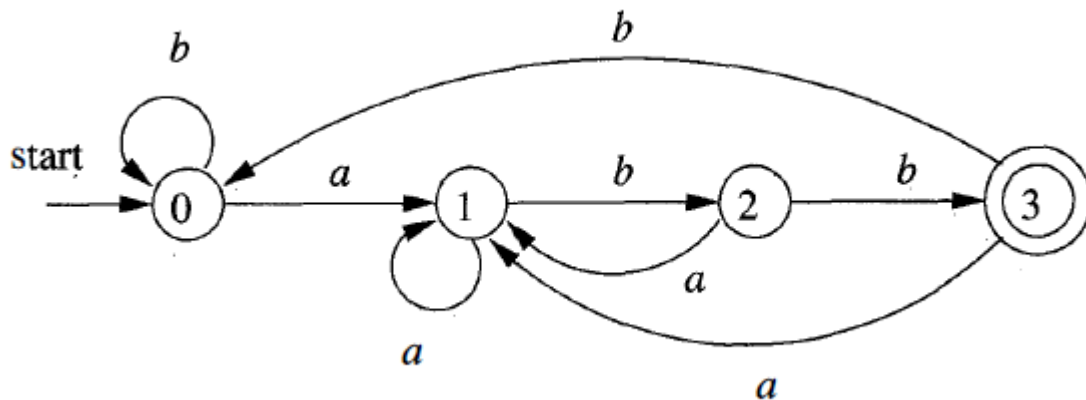


The language recognized by this NFA is  $aa^* | bb^*$

# Deterministic Finite Automata

- A deterministic finite automaton (DFA) is a special case of an NFA where:
  - There are no moves on input  $\epsilon$ , and
  - For each state  $q$  and input symbol  $a$ , there is exactly one edge out of  $q$  labeled  $a$ .

## Example



A DFA accepting  $(a|b)^*abb$



# Implementing/simulating a DFA

- ***Let us assume*** that the end of a string is marked with a special symbol (say ***eos***).
- The algorithm for recognition will be as follows: (an efficient implementation)

```
q = q0                                { start from the initial state }
c = nextchar  { get the next character from the input string }
while (c != eos) do                      { do until the end of the string }
    begin
        q = move(q,c)                    { transition function }
        c = nextchar
    end
if (q in F) then                          { if s is an accepting state }
    return "yes"
else
    return "no"
```

# Implementing/simulating an NFA

$Q = \varepsilon\text{-closure}(\{q_0\})$

$c = \text{nextchar}$

while ( $c \neq \text{eos}$ ) {

begin

$Q = \varepsilon\text{-closure}(\text{move}(Q, c))$

$c = \text{nextchar}$

end

{ set of all states that can be accessible from  $q_0$  by  $\varepsilon$ -transitions }

{ set of all states that can be accessible from a state in  $q$  by a transition on  $c$  }

if ( $Q \cap F \neq \Phi$ ) then      { if  $Q$  contains an accepting state }

return "yes"

else

return "no"

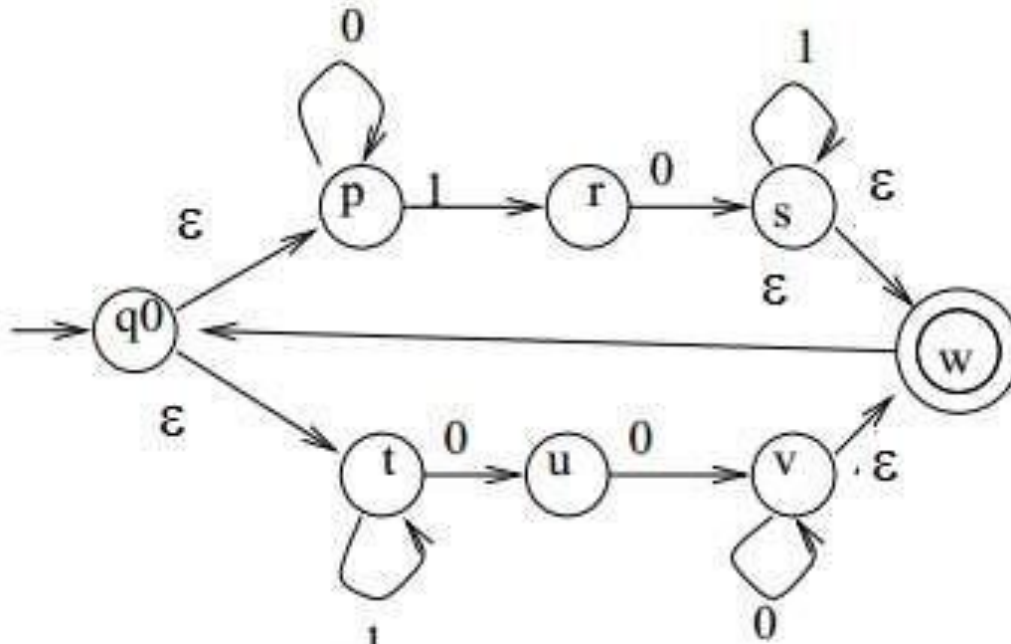
# Computing the $\epsilon$ -closure (Q)

- $\epsilon$ -closure( $q$ ): set of states reachable from some state  $q$  in  $Q$  on  $\epsilon$ -transitions alone.
- $move(q, c)$ : set of states to which there is a transition on input symbol  $c$  from some state  $q$  in  $Q$

```
push all states of  $Q$  onto stack;  
initialize  $\epsilon$ -closure( $Q$ ) to  $Q$ ;  
while ( stack is not empty ) {  
    pop  $q$ , the top element, off stack;  
    for ( each state  $u$  with an edge from  $q$  to  $u$  labeled  $\epsilon$  )  
        if (  $u$  is not in  $\epsilon$ -closure( $Q$ ) ) {  
            add  $u$  to  $\epsilon$ -closure( $Q$ );  
            push  $u$  onto stack;  
        }  
}
```

# Computing the $\epsilon$ -closure (Q) ...Example

Given an NFA below find  $\epsilon$ -closure (s)



$$\epsilon\text{-closure}(S) = \{s, w, q_0, p, t\}$$

i.e. All states that can be reached through  $\epsilon$ -transitions only

# From Regular Expression to NFA

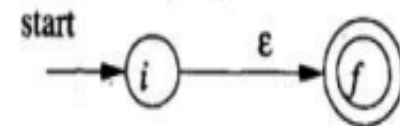
- For each kind of RE, there is a corresponding NFA
  - i.e. any regular expression can be converted to a NFA that defines the same language.
- The algorithm is syntax-directed, in the sense that it works recursively up the parse tree for the regular expression.
- For each sub-expression the algorithm constructs an NFA with a single accepting state.
- Algorithm: The ***McNaughton-Yamada-Thompson*** algorithm to convert a regular expression to a NFA.
  - **INPUT:** A regular expression  $r$  over alphabet  $\Sigma$ .
  - **OUTPUT:** An NFA  $N$  accepting  $L(r)$ .

# from Regular expressions to NFA...

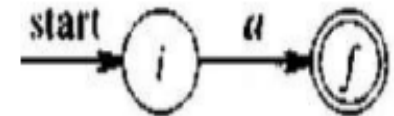
- **Method:** Begin by parsing  $r$  into its constituent sub-expressions.

Then, using rules (1) and (2) below, we construct NFA's for each of the basic symbols in  $r$

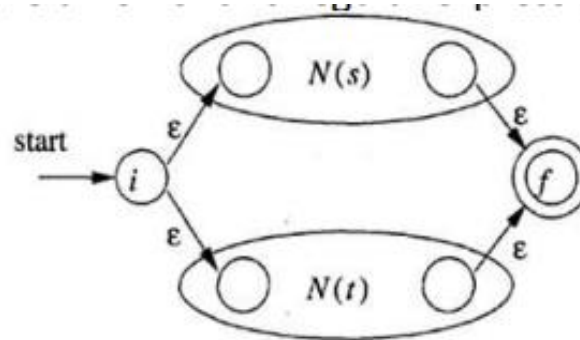
Rule1: For expression  $\varepsilon$  construct the NFA



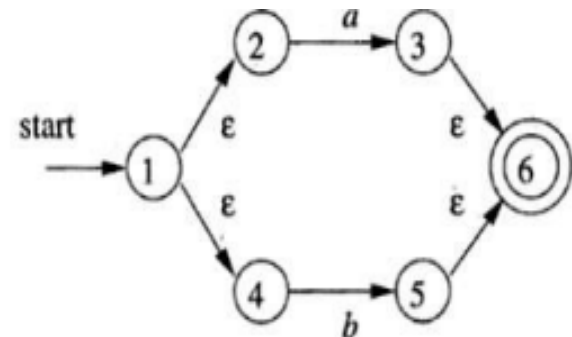
- Rule2: For any sub-expression  $a$  in  $\Sigma$ , construct the NFA



- Rule3: NFA for the union of two regular expressions

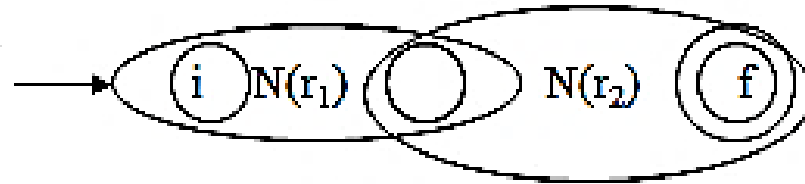


- Ex:  $a|b$

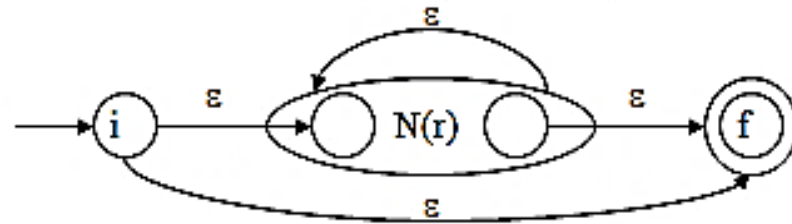


# from Regular expressions to NFA...

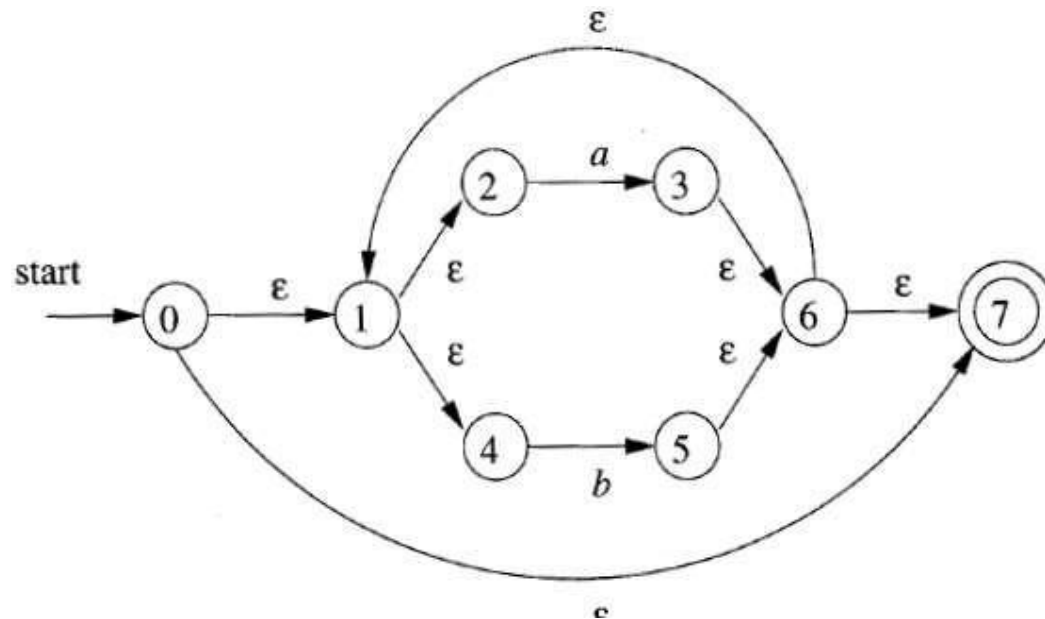
- Rule4: NFA for  $r_1r_2$



- Rule5: NFA for  $r^*$



- Example:  $(a|b)^*$



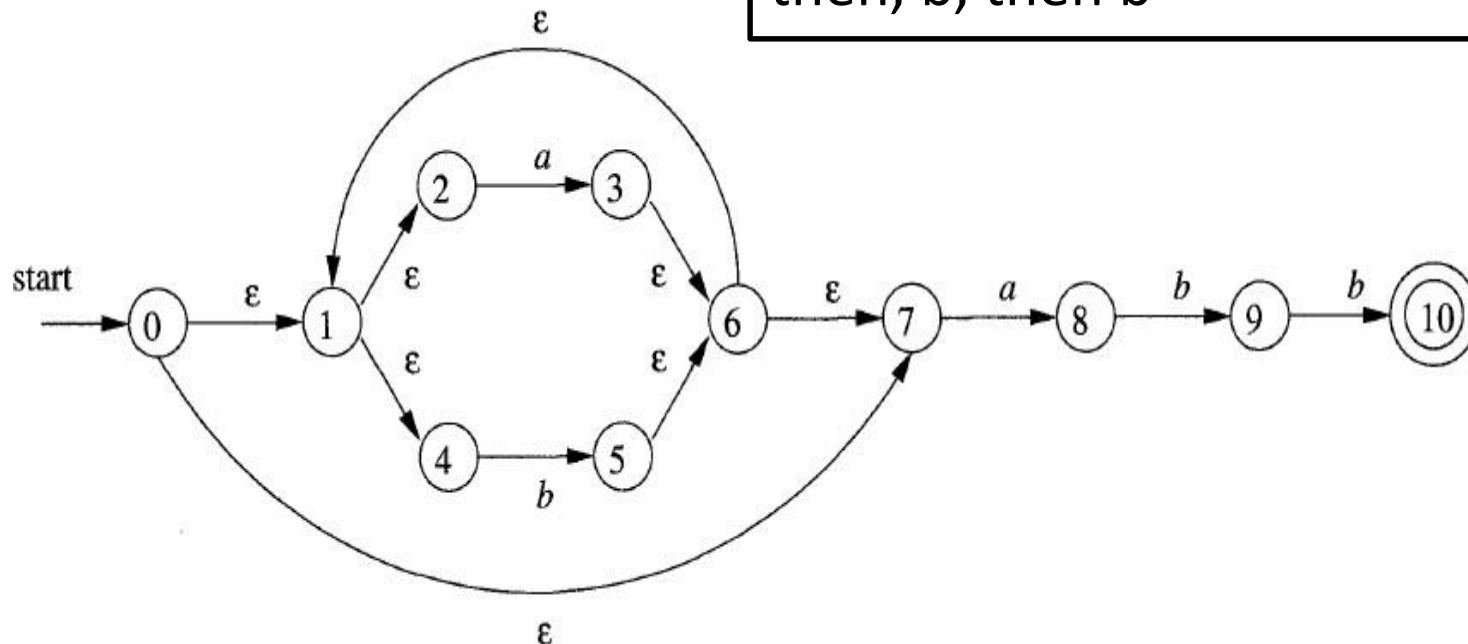
# from Regular expressions to NFA...

- **Example:** Constructing NFA for regular expression  
 $r = (a | b)^* abb$

*Step 1: constructing  $a | b$*

*Step 2: construct  $(a | b)^*$*

*Step 3: concatenate it with  $a$ ,  
then,  $b$ , then  $b$*





# Conversion of NFA to DFA

## ■ Why?

- DFA is difficult to construct directly from RE's
- NFA is difficult to represent in a computer program and inefficient to compute
- So,  $RE \rightarrow NFA \rightarrow DFA$

## ■ Conversion algorithm: **subset construction**

- The idea is that each DFA state corresponds to a set of NFA states.
- After reading input  $a_1a_2...a_n$ , the DFA is in a state that represents the subset  $Q$  of the states of the NFA that are reachable from the start state.
- **INPUT:** An NFA  $N$
- **OUTPUT:** A DFA  $D$  accepting the same language as  $N$ .
- **ALGORITHM:** ...
- The algorithm constructs a transition table  $D_{tran}$  for  $D$ .

# Subset Construction Algorithm

- Each state of  $D$  is set of NFA states and we construct **Dtran** so  $D$  will simulate “in parallel” all possible moves  $N$  can make on a given input string.

Algorithm: Note that  $q$  is a single state of  $N$ , while  $Q$  is a set of states of  $N$ .

$D_{\text{states}} := \varepsilon\text{-closure}(q_0)$  //  $q_0$  = initial state and they all are unmarked

**While** (there is an unmarked state  $Q$  in  $D_{\text{states}}$ ) **do**

**begin**

mark  $Q$ ;

**for** (each input symbol  $a$ ) **do**

**begin**

$U = \varepsilon\text{-closure}(\text{move}(Q, a))$ ;

**if** ( $U$  is not in  $D_{\text{states}}$ ) **then**

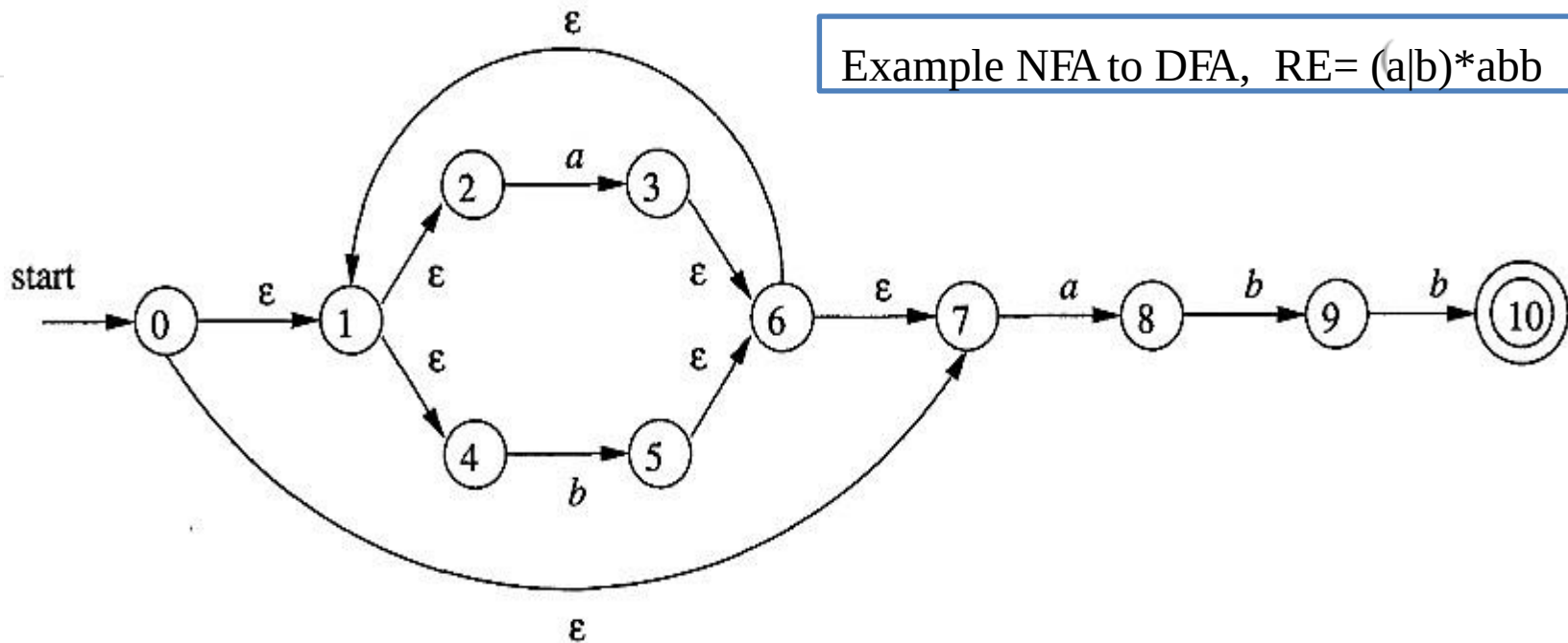
add  $U$  as an unmarked state to  $D_{\text{states}}$ ;

$D_{\text{tran}}[Q, a] = U$ ;

**end**

# Subset Construction Algorithm...

Example NFA to DFA, RE =  $(a|b)^*abb$



- The start state A of the equivalent DFA is  $\epsilon$ -closure(0),
  - $A = \{0, 1, 2, 4, 7\}$ ,
- Since these are exactly the states reachable from state 0 via a path all of whose edges have label  $\epsilon$ .
  - ***Note that a path can have zero edges, so state 0 is reachable from itself by an  $\epsilon$ -labeled path.***

# Subset Construction Algorithm...

- The input alphabet is {a, b}. Thus, our first step is to mark A and compute

$Dtran[A, a] = \epsilon\text{-closure}(\text{move}(A, a))$  and

$Dtran[A, b] = \epsilon\text{-closure}(\text{move}(A, b))$ .

- Among the states 0, 1, 2, 4, and 7, only 2 and 7 have transitions on a, to 3 and 8, respectively.
  - Thus,  $\text{move}(A, a) = \{3, 8\}$ . Also,  $\epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$ , so we call this set B,

$$Dtran[A, a] = \epsilon\text{-closure}(\text{move}(A, a)) = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$$

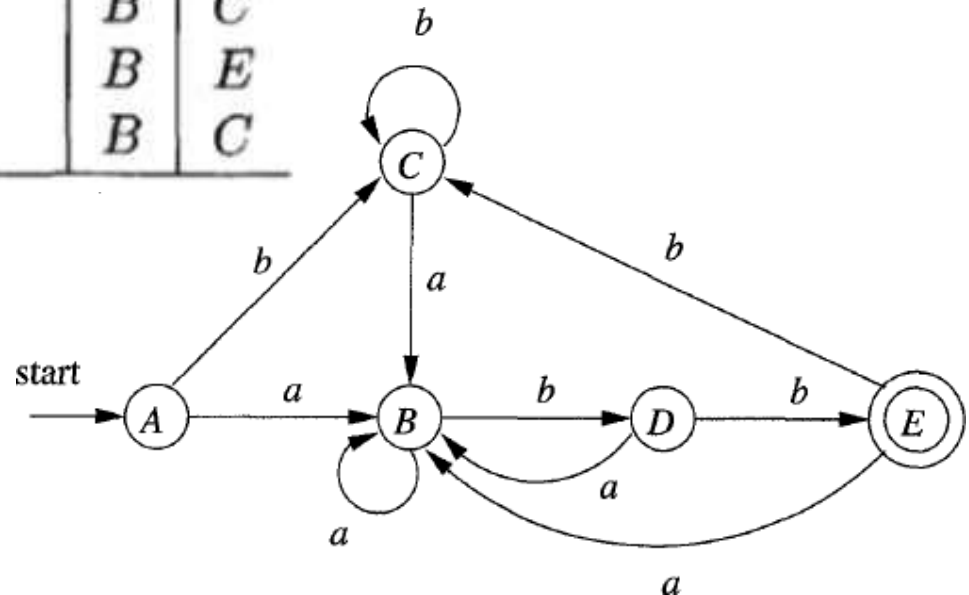
let  $Dtran[A, a] = B$

- compute  $Dtran[A, b]$ . Among the states in A, only 4 has a transition on b, and it goes to 5

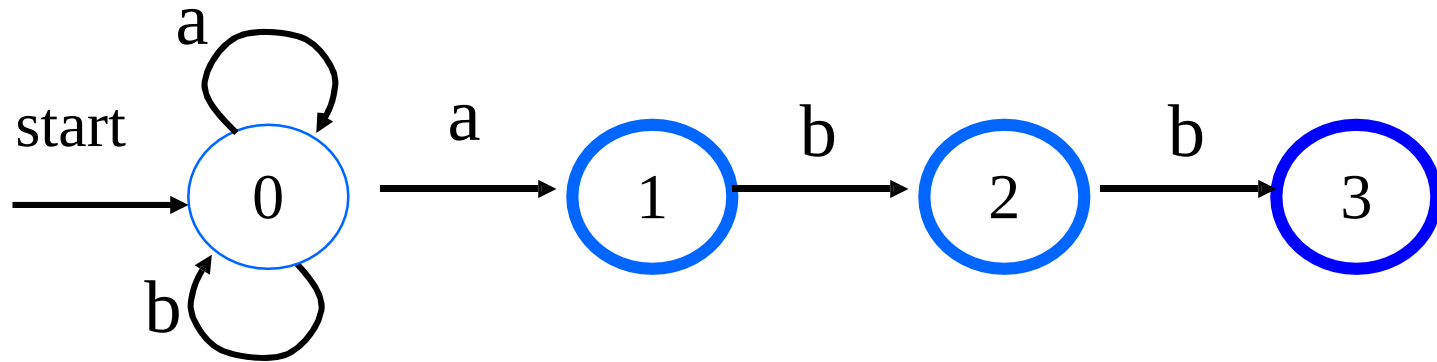
# Subset Construction Algorithm...

- $C = \text{Dtran}[A, b] = \epsilon\text{-closure}(\{5\}) = \{1, 2, 4, 5, 6, 7\}$
- If we continue this process with the unmarked sets B and C, we eventually reach a point where all the states of the DFA are marked

| NFA STATE              | DFA STATE | <i>a</i> | <i>b</i> |
|------------------------|-----------|----------|----------|
| {0, 1, 2, 4, 7}        | <i>A</i>  | <i>B</i> | <i>C</i> |
| {1, 2, 3, 4, 6, 7, 8}  | <i>B</i>  | <i>B</i> | <i>D</i> |
| {1, 2, 4, 5, 6, 7}     | <i>C</i>  | <i>B</i> | <i>C</i> |
| {1, 2, 4, 5, 6, 7, 9}  | <i>D</i>  | <i>B</i> | <i>E</i> |
| {1, 2, 4, 5, 6, 7, 10} | <i>E</i>  | <i>B</i> | <i>C</i> |



# NFA to DFA conversion: Example 2



$\epsilon$ -closure(0) = {0}

move(0,a) = {0,1}

move(0,b) = {0}

move({0,1}, a) = {0,1}

move({0,1}, b) = {0,2}

move({0,2}, a) = {0,1}

move({0,2}, b) = {0,3}

move({0,3}, a) = {0,1}

move({0,3}, b) = {0}

New states

A = {0}

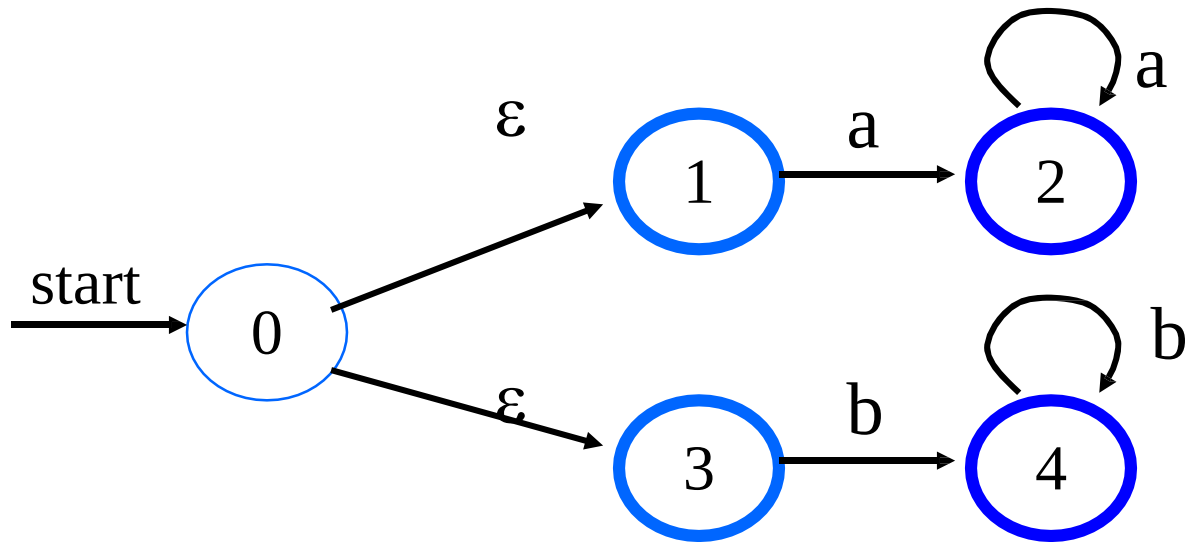
B = {0,1}

C = {0,2}

D = {0,3}

|   | a | b |
|---|---|---|
| A | B | A |
| B | B | C |
| C | B | D |
| D | B | A |

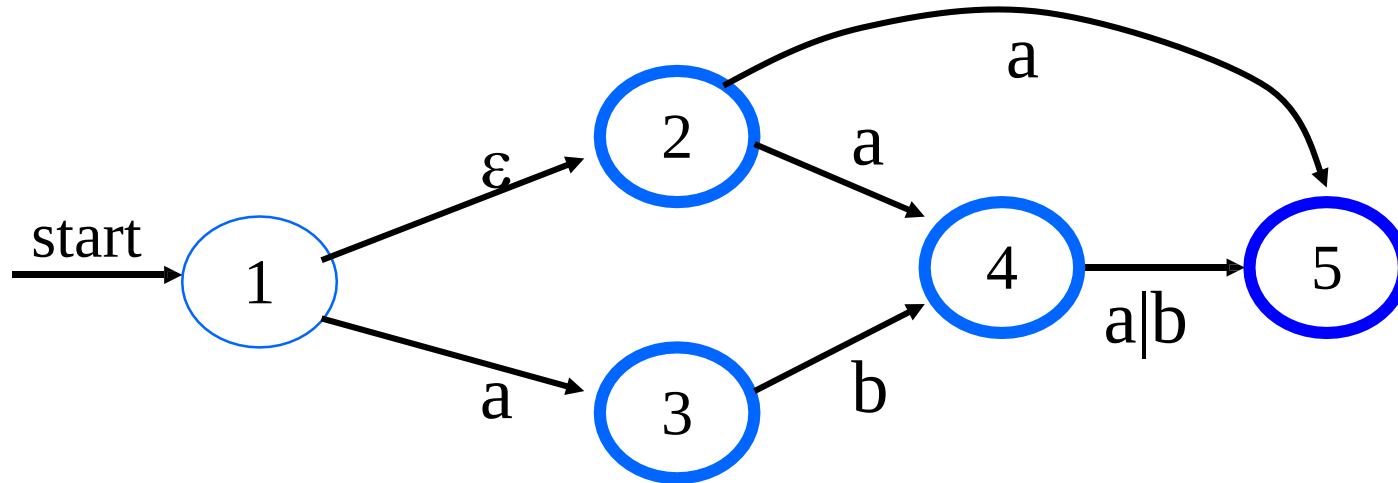
## NFA to DFA conversion: Example 3



Due to  $\epsilon$ -transitions, we must compute  $\epsilon$ -closure(S)

Example:  $\epsilon$ -closure (0) = {0,1,3}

# NFA to DFA conversion: Example 4



|          | a  | b  |
|----------|----|----|
| A{1,2}   | B  | -- |
| B{3,4,5} | D  | C  |
| C{4,5}   | D  | D  |
| D{5}     | -- | -- |

$D_{\text{states}} := \varepsilon\text{-closure}(1) = \{1,2\}$

$U := \varepsilon\text{-closure}(\text{move}(\{1,2\}, a)) = \{3,4,5\}$

Add  $\{3,4,5\}$  to  $D_{\text{states}}$

$U := \varepsilon\text{-closure}(\text{move}(\{1,2\}, b)) = \{\}$

$\varepsilon\text{-closure}(\text{move}(\{3,4,5\}, a)) = \{5\}$

$\varepsilon\text{-closure}(\text{move}(\{3,4,5\}, b)) = \{4,5\}$

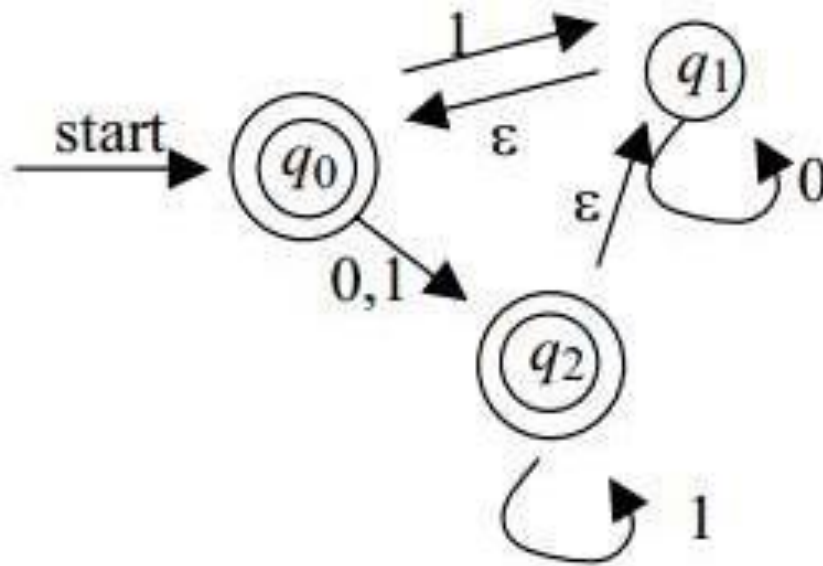
$\varepsilon\text{-closure}(\text{move}(\{4,5\}, a)) = \{5\}$

$\varepsilon\text{-closure}(\text{move}(\{4,5\}, b)) = \{5\}$

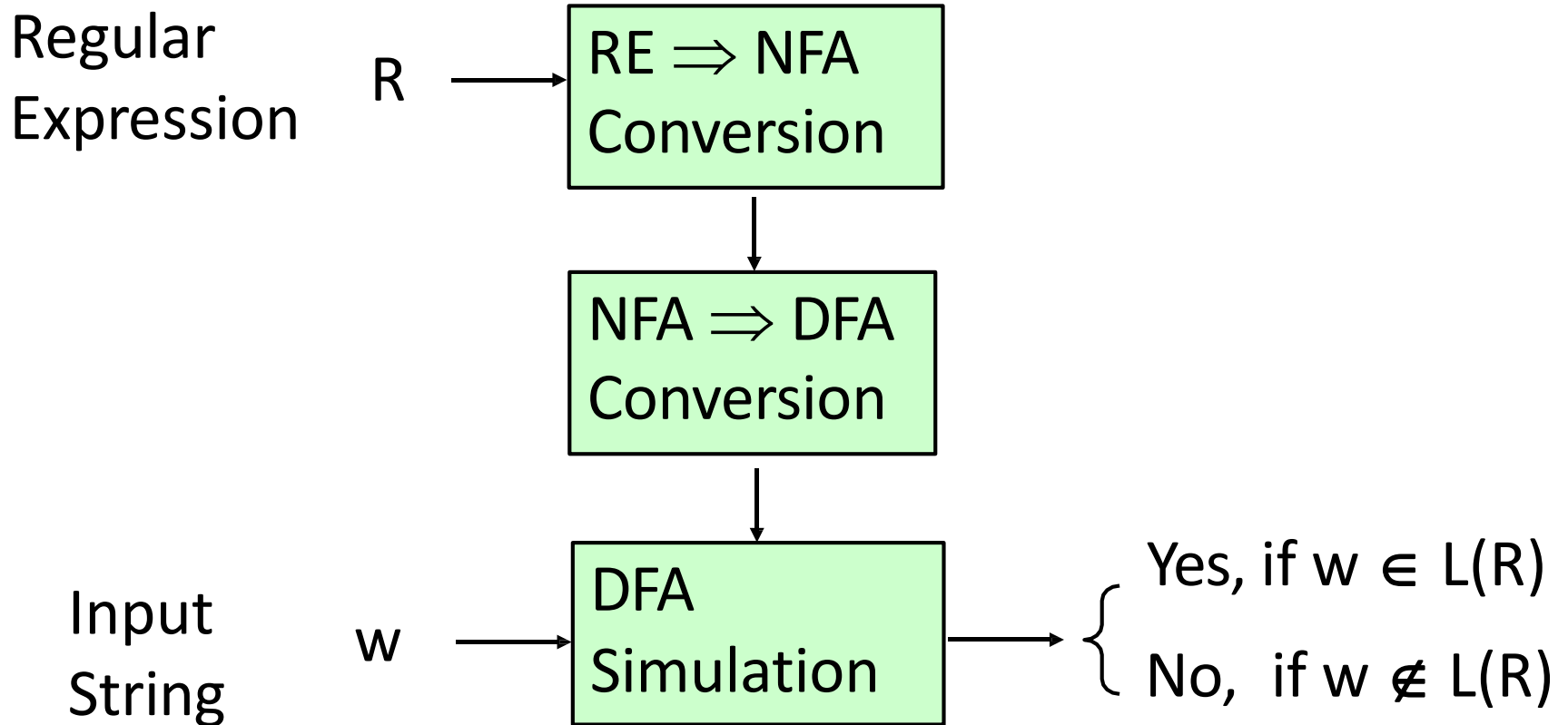


# NFA to DFA conversion: Exercise

Convert the -NFA shown below into a DFA, using subset construction

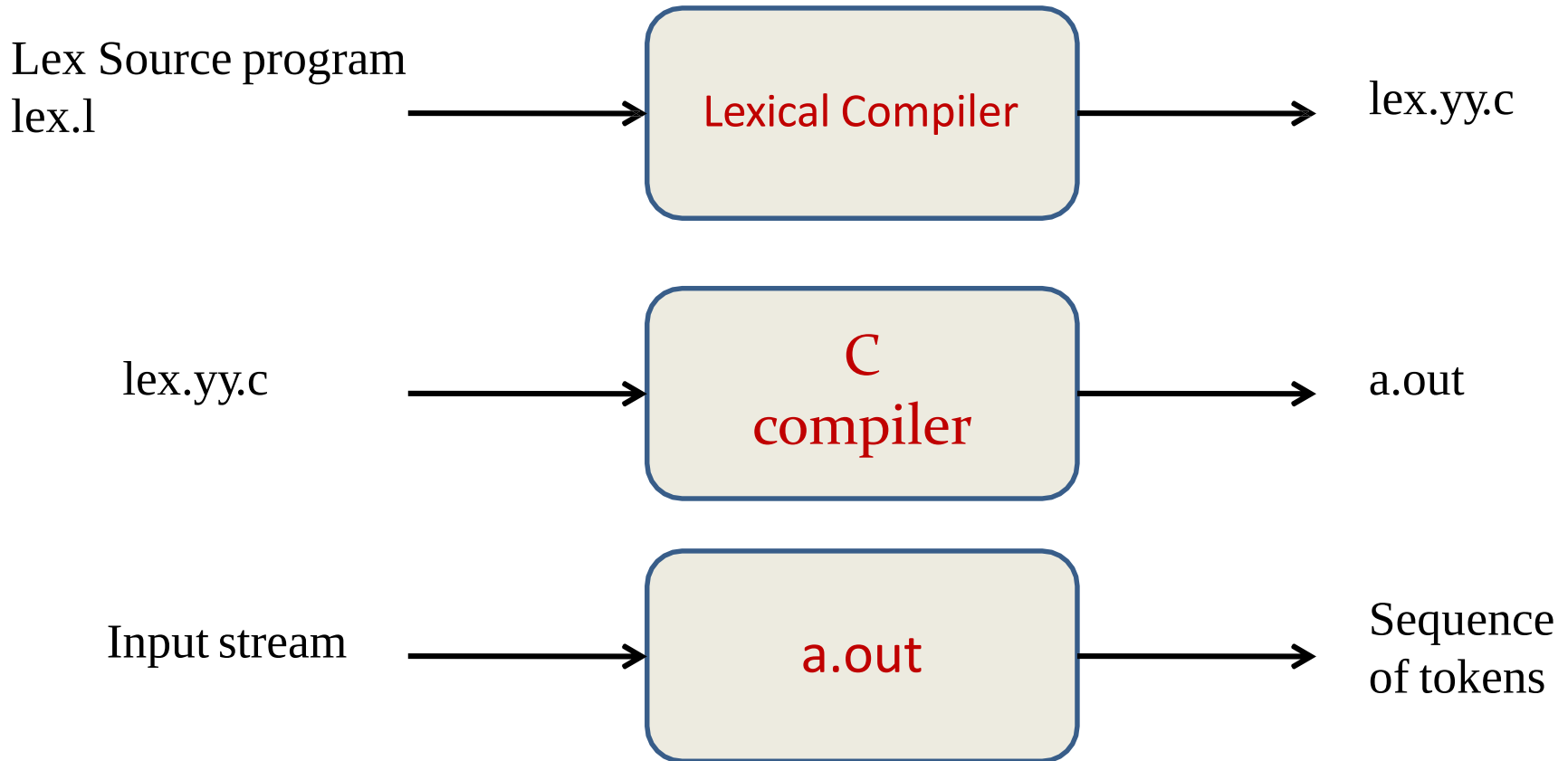


# Putting the Pieces Together



# Lexical Analyzer Generator - Lex

---



# Structure of Lex programs

declarations

%%

translation rules

%%

auxiliary functions



Pattern {Action}

That is:

%{

< C global variables, prototypes,  
comments >

%}



This part is copied as-is to  
the top of the generated  
C file

[DEFINITION SECTION]



Substitutions simplifies  
pattern matching

%%

[RULES SECTION]



Define how to scan and  
what action to take for  
each token

%%

< C auxiliary subroutines>



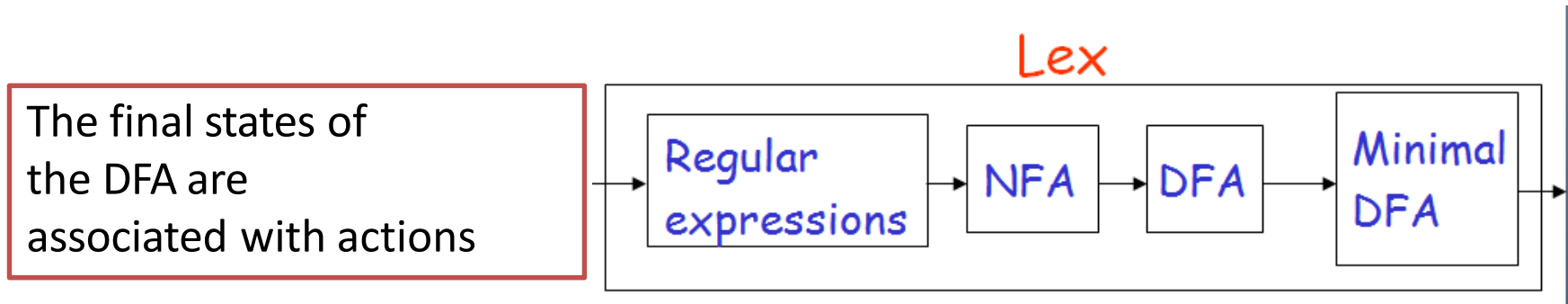
Any user code. For  
example, a main function  
to call the scanning  
function yylex().

24

# Structure of Lex programs

- Big difference from your previous coding experience
- Writing REs instead of the code itself
- Writing actions associated with each RE

## □ Internal Structure of Lex



# Example

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
%}  
/* regular definitions  
delim    [ \t\n]  
ws        {delim}+  
letter    [A-Za-z]  
digit     [0-9]  
id         {letter}({letter}|{digit})*  
number    {digit}+(\.{digit}+)?(E[+-]?{digit}+)?  
  
%%  
{ws}      { /* no action and no return */}  
if         {return(IF);}   
then       {return(THEN);}   
else       {return(ELSE);}   
{id}       {yylval = (int) installID(); return(ID); }   
{number}   {yylval = (int) installNum(); return(NUMBER);}   
...
```

## Example

```
Int installID() { /* funtion to install the lexeme, whose first  
character is pointed to by yytext, and whose length is  
yyleng, into the symbol table and return a pointer  
thereto */  
}
```

```
Int installNum() { /* similar to installID, but puts numerical  
constants into a separate table */  
}
```